More Info: Advance Search

Overview

Rudimentary boolean search algorithm with support for AND, OR, and NOT.

Usage

Users can use the feature by clicking on the “Advanced Search” button in the “View Recipes” page. Then they can select which section of text in the recipe they would like to search a keyword for by using the dropdown menu on the left side of the type/keyword pair. For example, a user can search for the keyword “dutch” in recipe titles by selecting the “Title” option in the dropdown and typing in “dutch” in the textbox next to it.

Users can add more terms to narrow down the search further by pressing the + button to the left of the keyword type dropdown. This will add a new type/keyword pair below the one they pressed the button on, and it will add a boolean operation dropdown between the new pair and the old pair. The three supported boolean operations are AND, OR, and NOT:

- AND: this represents a logical AND gate, so the two keywords immediately above and below it must both be “true” (in the recipe) in order for the recipe to appear in the search. For example: if a user searches for Title: “dutch” AND Title: “irish”, then only recipes with “dutch” and “irish” will be in the search.
- OR: this represents a logical OR gate, so the two keywords immediately above and below it can both be “true”, or either one but not the other can be “false” in order for the recipe to show up in the search. For example: if a user searches for Title: “dutch” OR Title: “irish”, then recipes with only “dutch”, only “irish”, and with both “dutch" and “irish” in the title will be in the search.
- NOT: this differs from the above two in that it doesn’t represent a basic gate, but instead acts more like set subtraction. So the search result will include any recipes that contain the above keyword but do not contain the below keyword. For example: if a user searches for Title: “dutch” NOT Title: “irish”, then only recipes with “dutch” and without “irish” will be in the search.

Users can add as many boolean operations as they wish, but adding more increases the likelihood that they get no results. Boolean operations are evaluated from top to bottom, as indicated in the UI. This means that normal boolean order of operations is not respected. So, searching for Title: “dutch” OR Title: “irish” AND Title: “pie” will result in recipes containing “dutch” “pie”, “irish” “pie”, and “dutch” “irish” “pie”, not in that order.

Users can also delete keywords, which will delete the boolean operator above it as well (unless it's the first keyword).

Implementation

All of the code for the advanced search feature is in the file `lib/view/screens/search_screen.dart`.

The main computations for the engine is in the `advancedFilter` function, which uses the two helper functions `advancedSearchHelper`, which parses a given Recipe for a keyword that is encoded in a Term and returns a truth value correlated to that Term, and `advancedFilterParser`, which calculates if a recipe should show up in the search based on all of the truth values for a given Recipe based on what `advancedSearchHelper` calculated. This system could be easily expanded to include more binary operations, if so desired.

The Advanced Search modal is built by the `AdvancedSearch` widget, which takes in a callback function that triggers when the “Search” button is pressed. This widget is where the `addSearchTerm` and `removeSearchTerm` functions live, and is also where the `Form` widget that makes up the keywords and boolean operators live. There are two more custom widgets used throughout, called `SearchTerm` and `BooleanTerm`, which simply act as helpers to build the `Form` when new terms are added, or old ones are deleted.

There is one custom type needed for this, which was used previously in this document. This type is the `Term` type, which is simply a type/keyword pair, but bundled into one type. The `Term` type is created using the constructor `Term(String term, String value)`. Each element can be accessed using `Term.term` and `Term.value`. There is also a `toString` method that returns a string formatted as `“$term: $value”`.